

Assignment 4

Shahriar Hasan

October 18, 2025

- 1 Manually draw a neural network having 3 hidden layers for classifying images of 10 English digits.

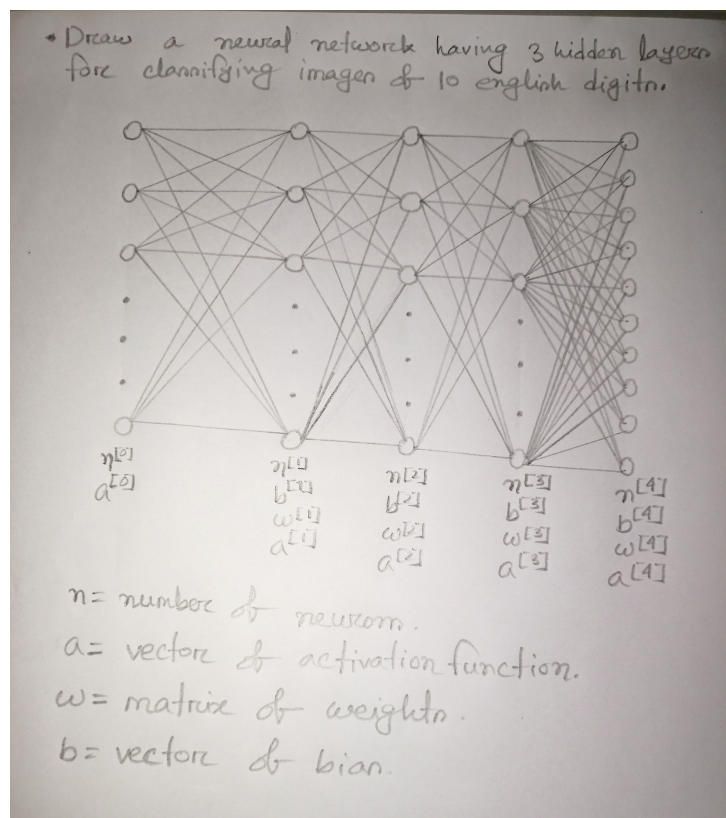


Figure 1: Neural network with 3 hidden layers for 10-digit classification.

2 Manually draw Forward Propagation Computation Graph

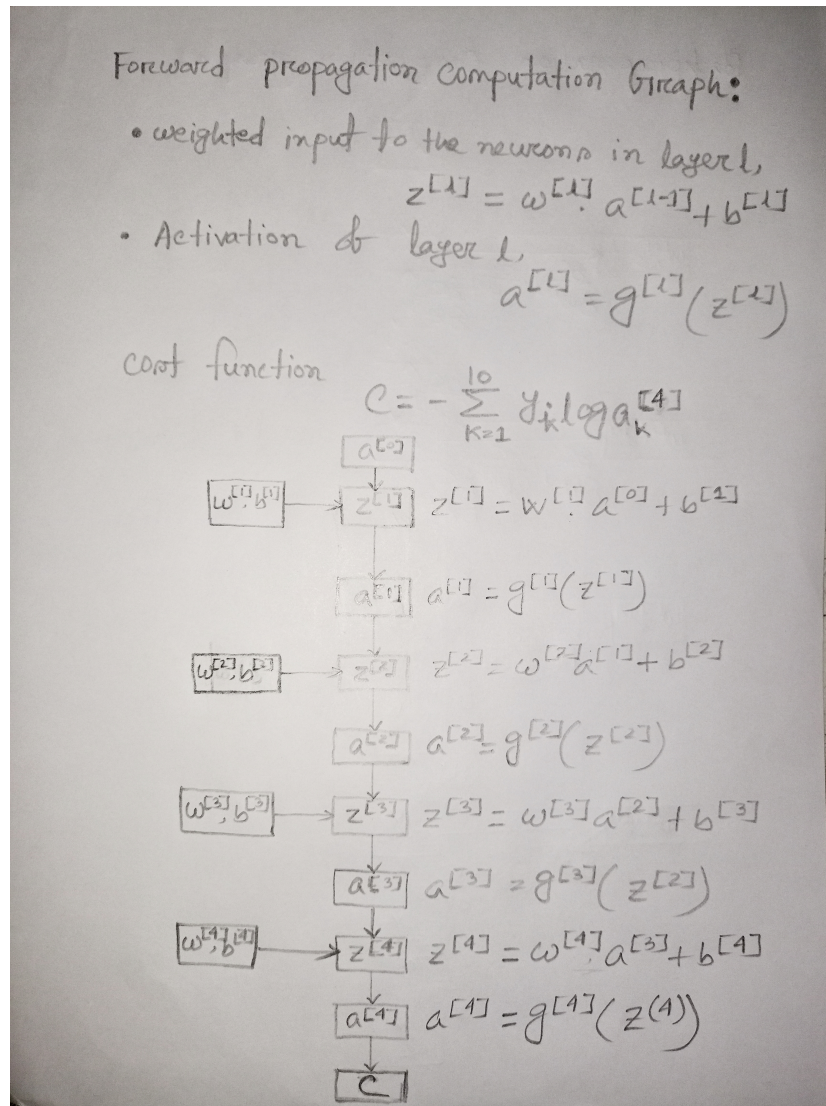


Figure 2: Forward propagation computation graph showing the flow of inputs through layers.

3 Manually draw Backpropagation Computation Graph

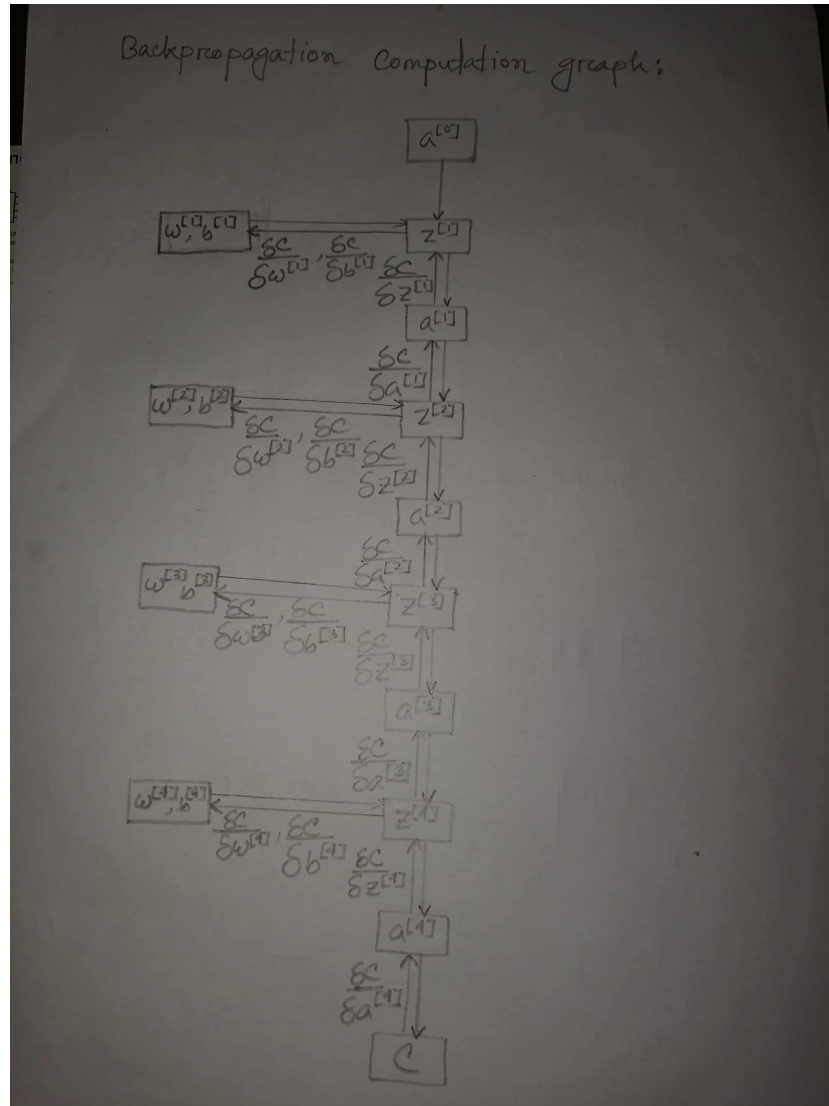


Figure 3: Backward propagation computation graph showing gradients flowing through the network.

4 Manually derive equations for updating weights of each hidden layer and output layer.

* Manually derive equation for updating weights of each hidden layer and output layer.

Update rule

$$w^{[4]} = w^{[4]} - \alpha \frac{\partial C}{\partial w^{[4]}}$$

where α is learning rate

$$w^{[3]} = w^{[3]} - \alpha \frac{\partial C}{\partial w^{[3]}}$$

$$w^{[2]} = w^{[2]} - \alpha \frac{\partial C}{\partial w^{[2]}}$$

$$w^{[3]} = w^{[3]} - \alpha \frac{\partial C}{\partial w^{[3]}}$$

$$w^{[4]} = w^{[4]} - \alpha \frac{\partial C}{\partial w^{[4]}}$$

Partial derivative of C with respect to parameters of output layer (layer-4)

$$\frac{\partial C}{\partial w^{[4]}} = \frac{\partial C}{\partial a^{[4]}} \cdot \frac{\partial a^{[4]}}{\partial z^{[4]}} \cdot \frac{\partial z^{[4]}}{\partial w^{[4]}}$$

Layer-3:

$$\frac{\partial C}{\partial w^{[3]}} = \frac{\partial C}{\partial a^{[4]}} \cdot \frac{\partial a^{[4]}}{\partial z^{[4]}} \cdot \frac{\partial z^{[4]}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial w^{[3]}}$$

layer 2:

$$\frac{\partial C}{\partial w^{[2]}} = \frac{\partial C}{\partial a^{[4]}} \cdot \frac{\partial a^{[4]}}{\partial z^{[4]}} \cdot \frac{\partial z^{[4]}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

layer-1:

$$\frac{\partial C}{\partial w^{[1]}} = \frac{\partial C}{\partial a^{[4]}} \cdot \frac{\partial a^{[4]}}{\partial z^{[4]}} \cdot \frac{\partial z^{[4]}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w^{[1]}}$$

Figure 4: Equations for updating weights

5 Train model using the MNIST digit dataset with the help of Tensorflow's `tf.GradientTape()`. Compare the performance of the model trained by `tf.GradientTape()` with Tensorflow's `model.fit()`

5.1 Objective

The goal of this study is to train a Convolutional Neural Network (CNN) on the MNIST dataset using **TensorFlow's** `tf.GradientTape()` for manual training, and to compare its performance with the standard `model.fit()` method. This comparison helps understand the differences between manual gradient computation and the built-in training loop.

5.2 Methodology

5.2.1 Dataset

- Dataset Used: MNIST handwritten digits
- Training Set: 60,000 images
- Test Set: 10,000 images
- Preprocessing: Normalization to [0,1] and reshaping to (28, 28, 1) for CNN input

```
x_train = x_train[..., tf.newaxis] / 255.0  
x_test = x_test[..., tf.newaxis] / 255.0
```

- Batching: Training batches of size 64, shuffled; test batches of size 64.

5.2.2 Model Architecture

A simple CNN was used for both training methods:

Layer	Output Shape
Conv2D(32, 3, ReLU)	$26 \times 26 \times 32$
MaxPooling2D	$13 \times 13 \times 32$
Flatten	5408
Dense(128, ReLU)	128
Dense(10)	10 logits

- Loss Function: `SparseCategoricalCrossentropy(from_logits=True)`
- Optimizer: Adam

5.2.3 Training Approaches

1. **GradientTape Training:** Manually compute gradients with `tf.GradientTape()` and apply using `optimizer.apply_gradients()`. Evaluate training and test accuracy each epoch.
2. **model.fit() Training:** Use built-in `fit()` with the same optimizer, loss, and batch size. Validate on the test set after each epoch.

5.3 Results

5.3.1 Accuracy after 5 Epochs

Method	Train Accuracy	Test / Validation Accuracy
<code>tf.GradientTape()</code>	0.9948	0.9833
<code>model.fit()</code>	0.9948	0.9863

5.3.2 Visualizations

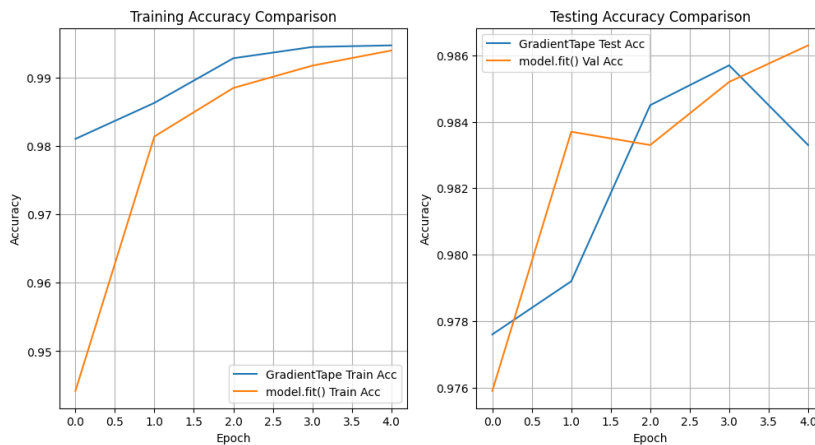


Figure 5: Comparison of CNN training using `tf.GradientTape()` and `model.fit()`. The plot shows both training and testing/validation accuracy over 5 epochs.

5.4 Analysis

- Both methods achieved nearly identical training accuracy, confirming that manual gradient computation with `GradientTape` is effective.
- Validation accuracy is slightly higher for `model.fit()`, likely due to internal optimizations like better handling of shuffling or updates.
- Accuracy curves show similar learning dynamics, indicating that both approaches are reliable for standard CNN training on MNIST.

Source: Github